IEAP Series 04 RStudio: Critical Analysis Report

Werner BINCHE, Elouan DELFIN, Vidusha THEBUWANA

2026-11-07

GitHub Repository: https://github.com/ElouanDef/IEAP-Series04-RStudio

1. Analytical Challenges & Modeling Choices

Our primary statistical challenge emerged during Part 3 when attempting to model heart rate recovery using a 5-parameter bi-exponential Non-Linear Least Squares (`nls`) regression. We encountered severe parameter unidentifiability, with the model failing to converge on physiologically realistic values (e.g., estimating an asymptotic resting heart rate of 0 bpm). We deliberately chose to prioritize the mono-exponential model instead. While the bi-exponential model yielded a marginally lower AIC, the mono-exponential approach was far more robust and provided biologically interpretable parameters (a realistic time constant and amplitude) without overfitting the noise. Additionally, wrangling the outputs of multiple `lm()` objects in Part 2 into a consolidated dataframe proved to be a meticulous data management task, reinforcing the need for clean code structure.

2. Technical Hurdles & Version Control

Adopting a modular Quarto architecture (`{{< include >}}`) successfully prevented merge conflicts but introduced pathing bugs. Code chunks developed interactively within the `sections/` directory utilized relative paths (`../data`) that broke upon master document compilation at the root level. We resolved this by standardizing root-relative pathing (`list.files("data")`) across all child documents.

Furthermore, configuring the Quarto YAML to output a GitHub-Flavored Markdown (`gfm`) file with fully rendered APA citations while preserving `.png` links proved highly complex. We discovered that enabling `embed-resources: true` for HTML output triggered an aggressive cleanup script that deleted the external image folder required by the Markdown file. We overcame this by removing the HTML dependency, isolating the `gfm` build format, and bypassing the default render cache via terminal execution (`quarto render --execute`), relying on our `.gitignore` to keep the commit history pristine.

3. Lessons Learned

This project reinforced that statistical fit must never supersede physiological plausibility; a model is only as useful as its interpretable parameters. Technically, we learned that reproducible environments require rigid path management and that automated build tools (like Quarto's render engine) possess opaque default behaviors. Successfully managing a modular repository demands overriding these defaults through CLI commands and strictly enforcing feature branches to protect the production environment.